Project Proposal: Transformer-Based Cancer Prognosis from Bulk RNA Sequencing Data

Course: CS 598 DLH - Deep Learning for Healthcare **Student:** Luis E. Fernandez de la Vara **Date:** November 2025 **Base Paper:** BulkRNABert: Cancer prognosis from bulk RNA-seq based language models

---

1. Introduction: Problem Statement

Clinical Challenge: Cancer remains one of the leading causes of death worldwide, with over 10 million deaths annually. Accurate prognosis—predicting patient survival time and treatment outcomes—is critical for personalized treatment planning, clinical trial stratification, and patient counseling. Traditional prognostic models rely on clinical variables (age, tumor stage, histology), but these explain only a fraction of outcome variability. The molecular revolution in cancer genomics has generated vast amounts of gene expression data through bulk RNA sequencing (RNA-seq), capturing transcriptome-wide snapshots of tumor biology.

Technical Problem: Bulk RNA-seq produces high-dimensional data (~20,000 genes per sample) with complex, non-linear relationships between gene expression patterns and clinical outcomes. Traditional machine learning approaches face several challenges: 1. **Curse of dimensionality:** More features (genes) than samples, causing overfitting 2. **Feature selection bias:** Manual gene selection discards potentially informative context 3. **Limited generalization:** Models trained on one cancer type fail to transfer to others 4. **Poor sample efficiency:** Require large datasets to achieve acceptable performance

Research Gap: While transformer-based language models have revolutionized NLP (BERT, GPT), their application to genomic data is nascent. Gene expression profiles can be conceptualized as "sentences" where genes are "words" and expression levels encode biological context. However, adapting transformers to continuous gene expression values (vs. discrete tokens) and leveraging self-supervised pre-training on massive unlabeled genomic datasets remains underexplored.

Significance: Successfully applying BERT-style pre-training to bulk RNA-seq could: - Enable transfer learning across cancer types, improving prognosis for rare cancers - Reduce sample requirements for clinical deployment (few-shot learning) - Discover novel gene interaction patterns not captured by pathway databases - Provide interpretable embeddings for biological discovery

---

2. Introduction: Citation to the Original Paper

Base Paper: Gélard, M., Richard, G., Pierrot, T., & Cournède, P. H. (2025). BulkRNABert: Cancer prognosis from bulk RNA-seq based language models. In *Proceedings of the 4th Machine Learning for Health Symposium* (Vol. 259, pp. 384-400). PMLR.

Also available as: - bioRxiv preprint: https://doi.org/10.1101/2024.06.18.599483 - Code: https://github.com/instadeepai/multiomics-open-research - Model: https://huggingface.co/InstaDeepAI/BulkRN

Paper Summary: BulkRNABert introduces the first BERT-style transformer encoder pre-trained on bulk RNA-seq profiles from The Cancer Genome Atlas (TCGA). The model uses **masked language modeling (MLM)**: randomly masking 15% of gene expression values and training the

model to reconstruct them from surrounding genomic context. Three pre-training strategies were evaluated: 1. **TCGA-only:** 10,327 cancer samples across 33 tumor types 2. **GTEx+ENCODE:** 17,382 normal tissue samples 3. **Combined:** 27,709 total samples

Key Results: - **Cancer-type classification:** 98.2% accuracy on 5-cohort TCGA test set (vs. 94.1% for baseline MLP) - **Survival prediction:** C-index of 0.72 on pan-cancer survival (vs. 0.65 for Cox proportional hazards) - **Transfer learning:** Pre-training on TCGA improves survival prediction on external GEO cohorts (+8% C-index boost) - **Sample efficiency:** With only 10% of training data, achieves 95% of full-data performance

The model demonstrates that self-supervised pre-training learns biologically meaningful gene representations capturing pathway co-regulation and tumor biology.

---

3. Methodology: Specific Approach

3.1 Reproduction Phase (Baseline Model)

Objective: Validate reproducibility by replicating original paper's performance on cancer classification and survival prediction.

3.1.1 Model Architecture

BulkRNABert is an encoder-only transformer following BERT's architecture:

```
Input: Gene expression vector (N genes)
       ↓
Gene Embedding Layer: Linear(N → 768)
       ↓
Positional Encoding: Learned embeddings (gene order matters!)
       ↓
Transformer Encoder:
  • 12 layers
  • 12 attention heads
  • Hidden size: 768
  • FFN intermediate size: 3072
  • Dropout: 0.1
  • Layer normalization
       ↓
[CLS] token embedding → Downstream tasks
```

Key Design Choices: - **Input representation:** Each sample is a vector of log-normalized TPM (Transcripts Per Million) values for ~20,000 protein-coding genes - **Gene ordering:** Genes are ordered by chromosomal position (maintains biological structure) - **[CLS] token:** Special token prepended to sequence; its final embedding used for classification - **Masking strategy:** 15% of genes randomly masked (80% replaced with [MASK], 10% random value, 10% unchanged)

3.1.2 Pre-Training Procedure

Objective: Learn general gene expression representations via self-supervised learning

Dataset: TCGA pan-cancer cohort - **Training:** 8,262 samples (80% of 10,327 total) - **Validation:** 1,033 samples (10%) - **Test:** 1,032 samples (10%) - **Cancer types:** 33 (BRCA, LUAD, COAD, etc.)

Preprocessing:

```
# Gene expression normalization
1. Raw counts → TPM (Transcripts Per Million)
2. Log-transform: log2(TPM + 1)
3. Z-score normalization per gene across samples
4. Filter genes: Remove <10 counts in >80% of samples
   → Final: ~19,000 genes per sample
```

Training Configuration: - **Loss:** Mean Squared Error (MSE) on masked gene reconstruction - **Optimizer:** AdamW (lr=1e-4, weight decay=0.01, $_1$=0.9, $_2$=0.999) - **Scheduler:** Linear warmup (10% steps) + linear decay - **Batch size:** 32 - **Epochs:** 100 (early stopping on validation loss, patience=10) - **Hardware:** 4× NVIDIA A100 (40GB), mixed precision (FP16) - **Time:** ~24 hours for TCGA pre-training

Masking Procedure:

```python
def mask_genes(expression_vector, mask_prob=0.15):
    """
    Randomly mask 15% of genes
    80%: replace with [MASK] token (0.0)
    10%: replace with random value from normal dist
    10%: keep original (helps model learn unmasked predictions)
    """
    mask_indices = random.sample(range(len(expression_vector)),
                                 int(len(expression_vector) * mask_prob))

    for idx in mask_indices:
        rand = random.random()
        if rand < 0.8:
            expression_vector[idx] = 0.0  # [MASK]
        elif rand < 0.9:
            expression_vector[idx] = np.random.randn()  # Random
        # else: keep original

    return expression_vector, mask_indices
```

3.1.3 Downstream Task 1: Cancer-Type Classification

Task: Predict cancer type (33 classes) from gene expression

Method: Fine-tune pre-trained BulkRNABert with classification head

```
[CLS] embedding (768-dim)
       ↓
Dropout (p=0.1)
       ↓
```

```
Linear (768 → 256)
       ↓
ReLU + LayerNorm
       ↓
Dropout (p=0.1)
       ↓
Linear (256 → 33)
       ↓
Softmax → Cancer type probabilities
```

Fine-tuning Config: - **Loss:** Cross-entropy - **Optimizer:** AdamW (lr=5e-5, smaller than pre-training) - **Batch size:** 16 - **Epochs:** 20 (early stopping, patience=5) - **Evaluation:** 5-fold stratified cross-validation on test set - **Metrics:** Accuracy, macro F1-score, per-class precision/recall

Success Criterion: Achieve >98% accuracy (matching original paper)

3.1.4 Downstream Task 2: Survival Prediction

Task: Predict time-to-event (death or last follow-up) from gene expression

Method: Fine-tune BulkRNABert with survival head using Cox proportional hazards loss

```
[CLS] embedding (768-dim)
       ↓
Dropout (p=0.1)
       ↓
Linear (768 → 128)
       ↓
Tanh activation
       ↓
Linear (128 → 1)  [No activation]
       ↓
Risk score (log-hazard ratio)
```

Loss Function: Negative log partial likelihood (Cox loss)

```python
def cox_loss(risk_scores, event_times, event_indicators):
    """
    Args:
        risk_scores: Model predictions (higher = worse prognosis)
        event_times: Survival times (days)
        event_indicators: 1=death, 0=censored
    """
    # Sort by survival time (descending)
    sorted_idx = np.argsort(-event_times)

    # Compute risk set at each event time
    log_risk = risk_scores[sorted_idx]
    events = event_indicators[sorted_idx]

    # Partial likelihood
```

```
    log_partial_likelihood = (
        log_risk * events -
        np.log(np.cumsum(np.exp(log_risk))) * events
    ).sum()

    return -log_partial_likelihood  # Negative for minimization
```

Fine-tuning Config: - **Optimizer:** AdamW (lr=1e-5, very small to avoid catastrophic forgetting) - **Batch size:** 64 (larger for stable gradient estimates) - **Epochs:** 50 - **Evaluation Metric:** Concordance Index (C-index) - C-index = P(model ranks patient A higher risk than B | A died before B) - Range: [0.5 (random), 1.0 (perfect)]

Success Criterion: C-index >0.70 on TCGA test set

3.2 Extension Analysis

Extension 1: Transfer Learning to External Cohorts Fine-tune TCGA-pretrained model on independent GEO datasets (different sequencing protocols, patient populations) to test generalization.

Extension 2: Rare Cancer Types Evaluate few-shot learning: fine-tune with only 10-50 samples from rare cancers (e.g., cholangiocarcinoma, uveal melanoma) not in TCGA.

Extension 3: Explainability via Attention Extract attention weights to identify which genes the model focuses on for prognosis. Compare to known prognostic biomarkers (e.g., TP53, EGFR).

Extension 4: Multi-Task Learning Simultaneously predict cancer type + survival in a single model. Hypothesis: Shared representations improve both tasks.

---

4. Methodology: Novelty/Relevance/Hypotheses to be Tested

4.1 Novelty

Computational Innovation: - **First BERT-style pre-training** on bulk RNA-seq (vs. single-cell or sequence-level) - **Continuous value masking:** Novel adaptation of discrete token masking to real-valued gene expression - **Chromosomal positional encoding:** Leverages spatial gene organization (vs. arbitrary ordering)

Clinical Relevance: - **Pan-cancer model:** Single model works across tumor types (vs. cancer-specific models) - **Transfer learning:** Pre-training on public data improves rare cancer prognosis - **Actionable predictions:** Survival risk scores inform treatment intensity decisions

4.2 Hypotheses

H1: Pre-Training Improves Sample Efficiency *Hypothesis:* BulkRNABert pre-trained on TCGA will achieve equivalent performance to from-scratch models using 50% fewer training samples. *Rationale:* Self-supervised learning on 10K+ samples should capture general gene co-expression patterns reusable for downstream tasks. *Test:* Plot C-index vs. training set size (10%, 25%, 50%, 75%, 100%). Pre-trained model should dominate baseline across all sizes (paired t-test, $p<0.05$).

H2: Combined Pre-Training (Cancer + Normal) Outperforms Cancer-Only *Hypothesis:* Model pre-trained on TCGA+GTEx+ENCODE (27,709 samples) will outperform TCGA-only (10,327 samples) on survival prediction. *Rationale:* Normal tissue data provides contrast, helping model learn cancer-specific dysregulation patterns. *Test:* Compare C-index of three pre-training strategies on held-out TCGA test set (Friedman test with post-hoc pairwise comparisons, $p<0.05$).

H3: Attention Recovers Known Prognostic Genes *Hypothesis:* Attention weights on [CLS] token will assign high importance to clinically validated prognostic genes (e.g., TP53 in breast cancer, EGFR in lung cancer). *Rationale:* If model learns meaningful biology, attention should align with domain knowledge. *Test:* 1. Extract attention weights for top 100 genes per cancer type 2. Compute overlap with literature-derived prognostic gene sets (GO terms, OncoKB) 3. Enrichment analysis: Hypergeometric test for over-representation ($p<0.01$)

H4: Transfer Learning Closes the Domain Gap *Hypothesis:* Fine-tuning TCGA-pretrained model on 100 GEO samples will match performance of training from scratch on 1000 GEO samples (10× data efficiency). *Rationale:* Pre-training captures universal gene expression patterns, reducing need for target-domain data. *Test:* - Baseline: Train from scratch on 1000 GEO samples → C-index = X - Transfer: Pre-train on TCGA, fine-tune on 100 GEO samples → C-index = Y - Success if Y X (non-inferiority test, margin=0.02)

4.3 Expected Outcomes

Quantitative: - **Reproduction:** Match paper's 98.2% cancer classification accuracy, 0.72 C-index - **Extension:** Achieve 0.68+ C-index on GEO cohorts (external validation) - **Few-shot:** 0.65+ C-index with only 50 rare cancer samples

Biological Discovery: - Identification of novel gene clusters via attention heatmaps - Validation that attention recovers ~60% of known OncoKB prognostic genes

Methodological: - Demonstration that chromosomal ordering matters (vs. random gene order ablation) - Quantification of pre-training data efficiency curves

---

5. Methodology: Ablations/Extensions Planned

5.1 Ablation Studies

A1: Pre-Training Dataset Comparison - **Variants:** TCGA-only, GTEx-only, TCGA+GTEx combined - **Impact:** Test whether normal tissue data helps cancer prognosis - **Evaluation:** C-index on survival, accuracy on cancer classification - **Hypothesis:** Combined > TCGA-only > GTEx-only

A2: Gene Ordering Strategies - **Variants:** 1. Chromosomal position (original) 2. Random shuffle 3. Functional grouping (KEGG pathways) 4. Expression variance ordering (high-variance genes first) - **Impact:** Assess whether positional encoding captures meaningful structure - **Evaluation:** Pre-training convergence speed, downstream performance - **Hypothesis:** Chromosomal > Functional > Random

A3: Masking Ratio Optimization - **Variants:** Mask 5%, 10%, 15% (original), 20%, 30% of genes - **Impact:** Trade-off between training signal and task difficulty - **Evaluation:** Pre-training

loss, downstream transfer quality - **Hypothesis:** 15% is optimal (following BERT NLP experiments)

A4: Model Size Scaling - **Variants:** 1. Small: 6 layers, 384 hidden, 6 heads (~20M params) 2. Base: 12 layers, 768 hidden, 12 heads (~80M params) [original] 3. Large: 24 layers, 1024 hidden, 16 heads (~300M params) - **Impact:** Determine if larger models justify computational cost - **Evaluation:** Performance vs. training time trade-off curve - **Hypothesis:** Diminishing returns beyond Base size for this dataset

5.2 Extensions Beyond Original Paper

E1: Multi-Omics Integration - **Extension:** Concatenate RNA-seq embeddings with DNA methylation data (using BulkRNABert + MethylBERT joint model from same lab) - **Impact:** Test if multimodal learning improves prognosis over RNA alone - **Rationale:** Epigenetic changes (methylation) complement transcriptomic dysregulation - **Evaluation:** C-index improvement, attention visualization across modalities

E2: External Validation on GEO Cohorts - **Extension:** Fine-tune on independent datasets: - GSE62254 (Breast cancer, n=248) - GSE39582 (Colorectal cancer, n=566) - GSE31210 (Lung adenocarcinoma, n=226) - **Impact:** Demonstrate real-world generalization beyond TCGA - **Challenges:** Different RNA-seq protocols, batch effects - **Mitigation:** Apply batch correction (ComBat) + domain adaptation techniques

E3: Interpretable Risk Scores via SHAP - **Extension:** Apply SHAP (SHapley Additive exPlanations) to survival predictions - **Impact:** Generate patient-specific gene importance scores for clinical interpretation - **Output:** "Top 10 genes driving this patient's high-risk score" - **Validation:** Compare SHAP-selected genes to clinician-identified prognostic markers

E4: Treatment Response Prediction - **Extension:** Fine-tune on TCGA samples with chemotherapy treatment data - **Task:** Predict response (complete/partial response vs. stable/progressive disease) - **Impact:** Move beyond prognosis to actionable treatment recommendations - **Dataset:** TCGA clinical annotations for ~3,000 patients with treatment records

E5: Rare Cancer Few-Shot Learning - **Extension:** Evaluate on ultra-rare TCGA cohorts with <100 samples: - Uveal melanoma (UVM, n=80) - Cholangiocarcinoma (CHOL, n=36) - Pheochromocytoma (PCPG, n=179) - **Method:** Meta-learning (MAML) or prototypical networks - **Impact:** Demonstrate few-shot capability for cancers with limited data

---

6. Data Access and Implementation Details: Access to Data

6.1 Datasets

All datasets are publicly available:

Dataset	Access	License	Samples	Purpose
TCGA (The Cancer Genome Atlas)	GDC Data Portal `https://portal.gdc.cancer.gov/`	Open access (dbGaP approval not needed for Level 3 data)	10,327 tumors33 cancer types	Pre-training & fine-tuning

Dataset	Access	License	Samples	Purpose
GTEx (Genotype-Tissue Expression)	GTEx Portal https://gtexportal.org/	Open access	11,688 normal tissues 54 tissue types	Pre-training (normal baseline)
ENCODE	ENCODE Portal https://www.encodeproject.org/	Open access	5,694 cell lines	Pre-training (additional normal)
GEO Cohorts	NCBI GEO https://www.ncbi.nlm.nih.gov/geo/	Public domain	Varies (100-1000 per cohort)	External validation

6.2 Data Collection Pipeline

Automated Download:

```python
# scripts/download_tcga.py

import gdc_client

def download_tcga_rnaseq(cancer_type='BRCA', data_type='htseq_counts'):
    """
    Download TCGA RNA-seq data via GDC API
    """
    query = {
        'project.project_id': f'TCGA-{cancer_type}',
        'files.data_type': 'Gene Expression Quantification',
        'files.experimental_strategy': 'RNA-Seq',
        'files.data_format': data_type
    }

    # Query GDC API
    files = gdc_client.query(query)

    # Download files
    gdc_client.download(files, output_dir=f'data/raw/{cancer_type}')

    return len(files)

# Download all 33 cancer types
for cancer in TCGA_CANCER_TYPES:
    download_tcga_rnaseq(cancer)
```

Preprocessing Pipeline:

```python
# scripts/preprocess_rnaseq.py

def preprocess_sample(raw_counts_file):
    """
    1. Raw counts → TPM normalization
```

```
    2. Log transform
    3. Gene filtering
    4. Z-score normalization
    """
    # Load raw counts
    counts = pd.read_csv(raw_counts_file, sep='\t', index_col=0)

    # TPM normalization (accounts for gene length + sequencing depth)
    gene_lengths = load_gene_lengths()  # From GENCODE annotation
    rpk = counts.div(gene_lengths, axis=0) * 1000  # Reads per kilobase
    tpm = rpk.div(rpk.sum(axis=0), axis=1) * 1e6  # Scale to 1 million

    # Log transform
    log_tpm = np.log2(tpm + 1)

    # Filter low-expression genes
    keep_genes = (counts > 10).sum(axis=1) > 0.2 * len(counts.columns)
    log_tpm = log_tpm.loc[keep_genes]

    # Z-score per gene
    z_scored = (log_tpm - log_tpm.mean(axis=1)) / log_tpm.std(axis=1)

    return z_scored
```

6.3 Clinical Annotations

Survival data access: - TCGA clinical data: Downloaded from GDC - Variables needed: - `vital_status` (alive/dead) - `days_to_death` or `days_to_last_follow_up` - `age_at_diagnosis` - `tumor_stage` (for stratification)

Data structure:

```
# Example clinical DataFrame
clinical_df = pd.DataFrame({
    'patient_id': ['TCGA-A1-A0SB', ...],
    'cancer_type': ['BRCA', ...],
    'event': [1, 0, 1, ...],  # 1=death, 0=censored
    'survival_days': [1825, 3000, 450, ...],
    'age': [56, 42, 68, ...],
    'stage': ['III', 'II', 'IV', ...]
})
```

6.4 Data Availability Statement

No restricted-access data used. All analyses rely on: 1. Level 3 TCGA data (processed, de-identified, publicly available) 2. GTEx v8 public release 3. ENCODE open-access datasets 4. GEO series with public accession numbers

Total storage: ~50GB (raw counts) + ~20GB (preprocessed)

7. Data Access and Implementation Details: Feasibility of the Computation

7.1 Computational Requirements

Hardware: - **GPU:** NVIDIA A100 (40GB) or V100 (32GB) — available via university HPC - **CPU:** 16+ cores for data preprocessing - **RAM:** 128GB (for loading full TCGA dataset in memory) - **Storage:** 300GB (data: 70GB, models: 10GB, results: 50GB, buffer: 170GB)

7.2 Estimated Compute Time

Task	Time (A100)	Justification
Data download & preprocessing	8 hours	Download TCGA (20GB) + GTEx (15GB), run preprocessing pipeline
Pre-training (TCGA-only)	24 hours	100 epochs × 8,262 samples ÷ batch 32 = 25,819 steps
Pre-training (TCGA+GTEx)	60 hours	3× more data (27,709 samples)
Fine-tuning (cancer classification)	2 hours	20 epochs on 8,262 samples, small learning rate
Fine-tuning (survival prediction)	4 hours	50 epochs, Cox loss requires careful optimization
Ablation studies	40 hours	4 ablations × 10 hours each (multiple training runs)
Extension experiments	30 hours	External cohorts, multi-task learning, SHAP analysis
Total	**168 GPU hours**	~7 days of continuous A100 time (parallelizable)

7.3 Feasibility Assessment

Highly Achievable: - **Pre-trained models available:** HuggingFace provides BulkRNABert checkpoints (skip pre-training if needed) - **Fast fine-tuning:** Downstream tasks take 2-4 hours each - **Efficient architecture:** 80M parameters (vs. 340M for BERT-Large in NLP)

Optimization Strategies: - **Mixed precision (FP16):** 2× speedup, 50% memory reduction - **Gradient accumulation:** Simulate larger batches on limited GPU memory - **Checkpoint resume:** Restart from failed runs without losing progress - **Use pre-trained weights:** Skip pre-training phase if time-constrained, focus on extensions

Backup Plan: - **Cloud resources:** Google Colab Pro ($10/month) provides A100 access if university HPC unavailable - **Smaller ablations:** Reduce hyperparameter search space if compute-limited - **Focus on reproduction:** If extensions prove too costly, thorough reproduction + 2-3 key extensions sufficient

8. Data Access and Implementation Details: Will Use Existing Code?

Yes, with significant modifications and extensions.

8.1 Base Code from Original Paper

Repository: https://github.com/instadeepai/multiomics-open-research **License:** Apache 2.0 (permissive, allows modification and redistribution)

What I'll Reuse: - Pre-trained BulkRNABert model weights (HuggingFace checkpoint) - TCGA data preprocessing pipeline (TPM normalization, gene filtering) - Model architecture definition (PyTorch code) - Downstream task examples (cancer classification, survival prediction)

What I'll Modify: - **Data loading:** Add GTEx and GEO dataset loaders - **Training loop:** Implement custom callbacks for logging, checkpointing - **Evaluation:** Add comprehensive metrics (calibration curves, survival curves) - **Ablations:** Implement gene ordering variants, masking ratio experiments

What I'll Build from Scratch: - **Transfer learning pipeline:** Fine-tuning on external GEO cohorts with domain adaptation - **Explainability module:** Attention visualization, SHAP analysis for gene importance - **Multi-task learning:** Joint cancer-type + survival prediction model - **Few-shot learning:** Meta-learning algorithms (MAML, Prototypical Networks) for rare cancers - **Evaluation dashboard:** Streamlit app for interactive result exploration

8.2 Additional Libraries

```
# requirements.txt (beyond original repo)

# Survival analysis
lifelines>=0.27.0          # Cox regression baseline, C-index computation
scikit-survival>=0.21.0    # Survival utilities

# Explainability
shap>=0.42.0               # SHAP values for gene importance
captum>=0.6.0              # PyTorch interpretability (attention viz)

# Data processing
scanpy>=1.9.0              # Single-cell tools (reusable for bulk)
pydeseq2>=0.4.0            # Differential expression

# Evaluation
matplotlib>=3.7.0
seaborn>=0.12.0
plotly>=5.15.0             # Interactive survival curves

# Web app
streamlit>=1.25.0          # Interactive dashboard

# Bioinformatics
biopython>=1.81
```

```
pybedtools>=0.9.0          # Genomic intervals

# Meta-learning (for few-shot extension)
learn2learn>=0.2.0         # MAML implementation
```

8.3 Code Attribution

All reused code will be: - Clearly commented with source citations in docstrings - Attribution in README and final report - Compliance with Apache 2.0 license (include NOTICE file) - Novel contributions separated into distinct modules

Example:

```python
# models/bulkrnabert.py

class BulkRNABert(nn.Module):
    """
    BERT-style transformer for bulk RNA-seq data.

    Architecture adapted from:
        Gélard et al. (2025). BulkRNABert: Cancer prognosis from
        bulk RNA-seq based language models. ML4H 2024.

    Original code:
        https://github.com/instadeepai/multiomics-open-research

    Modifications by Luis E. Fernandez de la Vara:
        - Added multi-task learning head
        - Implemented attention rollout for visualization
        - Extended to handle variable-length inputs
    """
    def __init__(self, num_genes=19000, hidden_size=768, num_layers=12):
        ...
```

---

9. Page Limit Compliance

This proposal: 9 pages (sections 1-8 + references) **Target:** 2-3 pages typical for course proposals **Strategy:** This is the comprehensive version. Will create 3-page condensed version focusing on: - Problem + paper summary (0.5 pages) - Methodology overview (1 page) - Hypotheses + extensions (1 page) - Data/feasibility (0.5 pages)

Appendices (not counted in page limit): - A. Detailed code snippets - B. Timeline/Gantt chart - C. Ablation study specifications - D. Extended related work

---

References

1. **Gélard, M., Richard, G., Pierrot, T., & Cournède, P. H.** (2025). BulkRNABert: Cancer prognosis from bulk RNA-seq based language models. In *Proceedings of the 4th Machine Learning for Health Symposium* (Vol. 259, pp. 384-400). PMLR.
2. **Devlin, J., Chang, M. W., Lee, K., & Toutanova, K.** (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT* (pp. 4171-4186).
3. **Weinstein, J. N., et al.** (2013). The Cancer Genome Atlas Pan-Cancer analysis project. *Nature Genetics*, 45(10), 1113-1120.
4. **Cox, D. R.** (1972). Regression models and life-tables. *Journal of the Royal Statistical Society: Series B*, 34(2), 187-202.
5. **Vaswani, A., et al.** (2017). Attention is all you need. In *NeurIPS* (pp. 5998-6008).
6. **Lundberg, S. M., & Lee, S. I.** (2017). A unified approach to interpreting model predictions. In *NeurIPS* (pp. 4765-4774). [SHAP]
7. **GTEx Consortium.** (2020). The GTEx Consortium atlas of genetic regulatory effects across human tissues. *Science*, 369(6509), 1318-1330.

Appendix A: Timeline

Week	Phase	Tasks	Deliverables
1	**Setup & Reproduction**	• Environment setup, GPU access verification • Download TCGA data (use preprocessed if available) • Load pre-trained BulkRNABert from HuggingFace • Fine-tune cancer classification task • Validate reproduction results	• Preprocessed TCGA data • Fine-tuned classification model • Reproduction validation: 98% accuracy

Week	Phase	Tasks	Deliverables
2	**Survival + Ablations**	• Implement Cox proportional hazards loss • Fine-tune survival prediction model • Validate C-index reproduction • Run 2 priority ablations: gene ordering + masking ratio • Generate survival curves (Kaplan-Meier)	• Survival model checkpoint • C-index >0.70 validation • Ablation results table • Survival plots by risk tertiles
3	**Transfer Learning + Explainability**	• Download GEO cohort (GSE62254 breast cancer) • Fine-tune on external data with batch correction • Extract attention weights for 3 cancer types (BRCA, LUAD, COAD) • Compute SHAP values for top samples • Gene enrichment analysis (compare to OncoKB)	• External validation C-index • Attention heatmaps • SHAP gene importance plots • Enrichment test results

Week	Phase	Tasks	Deliverables
4	**Analysis & Finalization**	• Statistical hypothesis testing (H1-H4) • Create all figures for report (8 total) • Write final report (8-10 pages) • Record 5-minute presentation video • Finalize code documentation and README • Prepare submission package	• Final report PDF • Presentation video • Complete GitHub repo • Code with documentation • All figures and results

Accelerated Timeline Notes: - **Skip pre-training:** Use HuggingFace pre-trained model (saves 24 GPU hours) - **Focus on core extensions:** Prioritize transfer learning + explainability over all ablations - **Streamlined validation:** Test on 1 GEO cohort instead of 3 - **Parallel execution:** Run ablations while waiting for model fine-tuning

---

Appendix B: Risk Mitigation

Risk	Likelihood	Impact	Mitigation
GPU access delays	Medium	High	• Use HuggingFace pre-trained weights (skip pre-training) • Colab Pro backup • Focus on fine-tuning (2-4 hours per task)
Data download failures	Low	Medium	• Use GDC Data Transfer Tool (resumable downloads) • Keep raw data separate from preprocessed
Reproduction discrepancies	Medium	Medium	• Contact authors via GitHub issues • Test multiple random seeds • Document all deviations
External cohort incompatibility	Medium	Low	• Apply robust batch correction (ComBat, Limma) • Report results with/without correction

Risk	Likelihood	Impact	Mitigation
Insufficient time for all extensions	High	Low	• Prioritize: Reproduction > Ablations > 2 key extensions • Drop few-shot learning if needed (most complex)

Appendix C: Expected Figure Outputs

1. **Figure 1:** Model architecture diagram (Input → Transformer → Outputs)
2. **Figure 2:** Pre-training loss curves (TCGA vs. TCGA+GTEx)
3. **Figure 3:** Confusion matrix for 33-cancer classification
4. **Figure 4:** Kaplan-Meier survival curves stratified by risk score tertiles
5. **Figure 5:** C-index vs. training set size (sample efficiency)
6. **Figure 6:** Attention heatmap for top 50 genes in breast cancer
7. **Figure 7:** SHAP beeswarm plot showing gene importance distribution
8. **Figure 8:** Transfer learning results on GEO cohorts (bar chart)